

IEW (Issue/Execute/Writeback) 阶段规范文档

1. 概述

1.1 设计目的

IEW 阶段是 gem5 O3 CPU 流水线后端的乱序执行核心。它接收来自 Rename（重命名）阶段的已重命名指令，将其分发到指令队列（IQ）和加载存储队列（LSQ），在操作数就绪时将指令发射到功能单元执行，将执行结果写回物理寄存器文件，并将已完成的指令转发到 Commit（提交）阶段。

1.2 三个子阶段

子阶段	职责
Dispatch（分发）	接收 Rename 输出的指令，按类型路由到 IQ（非内存指令）或 LSQ（内存指令），在队列满时通过阻塞机制施加反压
Execute（执行）	从 IQ 获取就绪指令并发射到功能单元，直接执行非内存指令，将内存操作委托给 LSQ，校验分支预测准确性，检测内存序违规
Writeback（写回）	唤醒依赖指令，在记分板中标记物理寄存器就绪，将已完成指令转发到 Commit 阶段

1.3 源代码文件

文件	描述
src/cpu/o3/iew.hh	IEW 类声明、状态枚举、接口定义
src/cpu/o3/iew.cc	IEW 实现：tick、dispatch、execute、writeback
src/cpu/o3/inst_queue.hh	InstructionQueue 类、IQUnit、FUCompletion 事件
src/cpu/o3/inst_queue.cc	IQ 实现：插入、调度、唤醒依赖、squash
src/cpu/o3/lsq.hh	LSQ 类、LSQRequest、DcachePort
src/cpu/o3/lsq.cc	LSQ 实现
src/cpu/o3/lsq_unit.hh/cc	每线程 LSQUnit 实现
src/cpu/o3/fu_pool.hh/cc	功能单元池：分配、延迟、流水线
src/cpu/o3/dep_graph.hh	DependencyGraph：每物理寄存器的依赖消费者链表
src/cpu/o3/scoreboard.hh	Scoreboard：物理寄存器就绪状态跟踪
src/cpu/o3/comm.hh	阶段间通信数据结构

2. 整体架构

2.1 IEW 类定义

IEW

- `_status`: Active / Inactive (整体阶段状态)
- `dispatchStatus[MaxThreads]`: Running / Blocked / Idle / StartSquash / Squashing / Unblocking
- `exeStatus`: StageStatus (全局执行状态)
- `wbStatus`: StageStatus (全局写回状态)

核心组件

- `instQueue`: InstructionQueue (非内存指令队列)
- `ldstQueue`: LSQ (加载/存储队列)
- `scoreboard`: Scoreboard* (物理寄存器就绪状态)
- `fuPools`: vector<FUPool*> (功能单元池列表)

缓冲结构

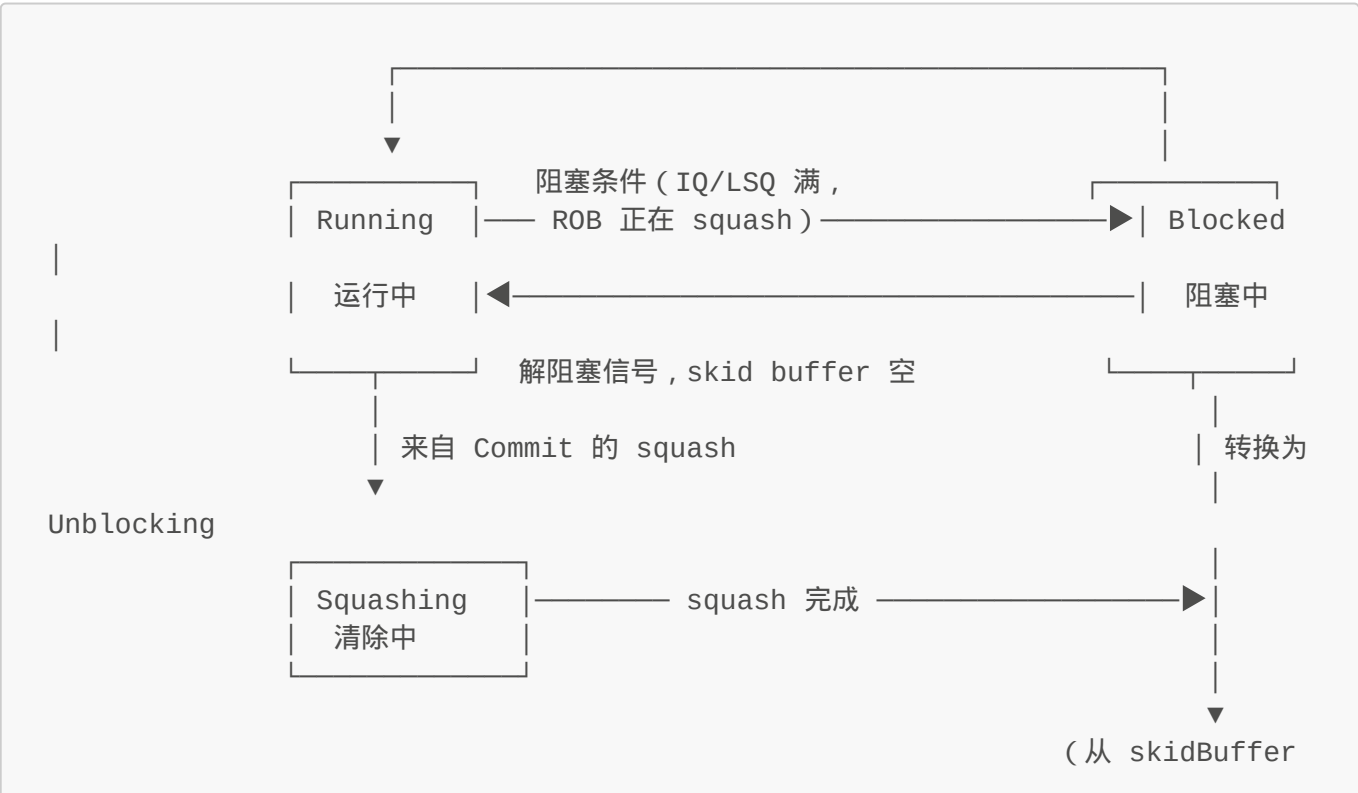
- `insts[MaxThreads]`: queue<DynInstPtr> (接收 Rename 输出)
- `skidBuffer[MaxThreads]`: queue<DynInstPtr> (溢出缓冲)
- `issueToExecQueue`: TimeBuffer<IssueStruct> (内部 IQ→Execute)
- `iewQueue`: TimeBuffer<IEWStruct> (输出到 Commit)

配置参数

- `dispatchWidth`: unsigned (每周期最大分发指令数)
- `issueWidth`: unsigned (每周期最大发射指令数)
- `wbWidth`: unsigned (每周期最大写回指令数)
- `renameToIEWDelay`: Cycles
- `commitToIEWDelay`: Cycles
- `issueToExecuteDelay`: Cycles

2.2 Dispatch 状态机

每个线程的 `dispatchStatus` 状态转换如下：



分发	指令，然后切换为 Running)
----	--------------------

全局 `exeStatus` 和 `wbStatus` 状态：

- **Running**：正在执行/写回指令
- **Idle**：无指令可处理
- **Squashing**：因 squash 清除指令（仅 `exeStatus`）
- **Unblocking**：正在从阻塞状态恢复（仅 `dispatchStatus`）

2.3 Probe 探测点

探测点名称	触发时机
<code>Dispatch</code>	指令被分发到 IQ/LSQ 时
<code>Mispredict</code>	检测到分支预测错误时
<code>Execute</code>	指令开始执行时
<code>ToCommit</code>	指令执行完成并准备提交时

3. 阶段间通信接口

3.1 输入：Rename 队列 (`RenameStruct`)

通过 `fromRename` 连线接收，延迟为 `renameToIEWDelay` 个周期。

```
struct RenameStruct {
    int size;
    DynInstPtr insts[MaxWidth]; // 已重命名的指令
};
```

3.2 输入：Commit 信号 (`CommitComm`，通过 `TimeStruct`)

通过 `fromCommit` 连线接收，延迟为 `commitToIEWDelay` 个周期。

字段	消费者	作用
<code>squash</code>	F, D, R, I	清除所有比指定序号更年轻的指令
<code>robSquashing</code>	F, D, R, I	ROB 正在 squash 中（暂停分发）
<code>doneSeqNum</code>	F, I	已提交/已清除指令的序列号
<code>nonSpecSeqNum</code>	I	调度一条非推测指令
<code>strictlyOrdered</code>	I	重放严格有序加载指令
<code>strictlyOrderedLoad</code>	I	需要重放的严格有序加载指令

3.3 输出：IEW 队列（IEWStruct）到 Commit

通过 toCommit 连线发送（零延迟，同周期到达）。

```
struct IEWStruct {
    int size;
    DynInstPtr insts[MaxWidth];           // 已完成的指令
    DynInstPtr mispredictInst[MaxThreads];
    Addr mispredPC[MaxThreads];
    InstSeqNum squashedSeqNum[MaxThreads];
    std::unique_ptr<PCStateBase> pc[MaxThreads];
    bool squash[MaxThreads];
    bool branchMispredict[MaxThreads];
    bool branchTaken[MaxThreads];
    bool includeSquashInst[MaxThreads];
};
```

3.4 输出：反向时间缓冲（IewComm，通过 TimeStruct）

通过 toRename 和 toFetch 连线发送（零延迟）。

字段	消费者	作用
freeIQEntries	R	IQ 空闲条目数
freeLQEntries	R	LQ 空闲条目数
freeSQEntries	R	SQ 空闲条目数
dispatched	R	本周周期分发的指令数
dispatchedToLQ	R	本周周期分发到 LQ 的加载数
dispatchedToSQ	R	本周周期分发到 SQ 的存储数
iqCount	F	IQ 使用量计数
ldstqCount	F	LSQ 使用量计数
iewBlock[tid]	R	阻塞 Rename（IQ/LSQ 已满）
iewUnblock[tid]	R	解除 Rename 阻塞（资源可用）

3.5 内部：Issue-to-Execute 队列（IssueStruct）

IQ 调度与执行之间的内部时间缓冲。

```
struct IssueStruct {
    int size;
    DynInstPtr insts[MaxWidth]; // 准备执行的指令
};
```

该缓冲的连线访问偏移量为 `-issueToExecuteDelay`，即在第 `N` 周期调度的指令将在第 `N + issueToExecuteDelay` 周期可供执行。

3.6 通信延迟汇总

路径	延迟	实现机制
Rename → IEW（指令）	<code>renameToIEWDelay</code> 周期	<code>renameQueue</code> 时间缓冲
Commit → IEW（squash/提交）	<code>commitToIEWDelay</code> 周期	<code>timeBuffer</code> 反向连线
IEW → Commit（已完成）	0 周期（同周期）	<code>iewQueue</code> 正向连线
IQ 调度 → Execute	<code>issueToExecuteDelay</code> 周期	<code>issueToExecQueue</code> 内部缓冲

4. Tick 主函数

4.1 每周期执行顺序

```
tick()
|
| 1. 重置每周期状态
|   wbNumInst = 0, wbCycle = 0
|   wroteToTimeBuffer = false, updatedQueues = false
|
| 2. LSQ tick
|   ldstQueue.tick()
|
| 3. 对 Rename 传来的指令按线程分类
|   sortInsts() — 按线程划分到 insts[tid]
|
| 4. 处理 FU 池释放
|   fu_pool->processFreeUnits() 对每个 FU 池
|
| 5. 对每个活跃线程（检查信号 + 分发）
|   checkSignalsAndUpdate(tid)
|   dispatch(tid)
|
| 6. 执行（若非 squash 状态）
|   executeInsts()
|
| 7. 写回（若非 squash 状态）
|   writebackInsts()
|
| 8. 调度就绪指令（若非 squash 状态）
|   instQueue.scheduleReadyInsts()
|
| 9. 推进内部时间缓冲
|   issueToExecQueue.advance()
|
| 10. 更新 exeStatus / 判断活跃度
|
```

```

├─ 11. 写回 store (利用剩余带宽)
    |   ldstQueue.writebackStores()
    |
├─ 12. 处理 Commit 的提交完成信号
    |   ldstQueue.commitStores(doneSeqNum, tid)
    |   ldstQueue.commitLoads(doneSeqNum, tid)
    |   instQueue.commit(doneSeqNum, tid)
    |   处理 nonSpecSeqNum / strictlyOrderedLoad
    |
├─ 13. 广播空闲条目数
    |   toFetch->iiewInfo[tid].iqCount / ldstqCount
    |   toRename->iiewInfo[tid].freeIQEntries / freeLQEntries / freeSQEntries
    |
└─ 14. 更新整体状态
    |   updateStatus()

```

4.2 关键执行约束

- Dispatch、Execute、Writeback 三个子阶段在**同一周期内依次顺序执行**，前一个子阶段的输出可以被后一个子阶段立即消费
- Execute 和 Writeback 在 `exeStatus == Squashing` 时被跳过
- `scheduleReadyInsts()` 在 Execute 之后执行，调度的是**下一个周期**将要发射的指令

5. Dispatch (分发) 子阶段

5.1 dispatch() 入口逻辑

```

dispatch(tid)
├─ Running/Idle ─► dispatchInsts(tid) (从 insts[tid] 分发)
├─ Unblocking ─► dispatchInsts(tid) (从 skidBuffer[tid] 分发)
                  如果 Rename 有新指令: skidInsert(tid)
                  unblock(tid)
└─ Squashing ─► 不分发指令, 等待 squash 完成

```

5.2 dispatchInsts() 算法

输入: 线程 ID `tid`
 来源: Unblocking 状态下取 skidBuffer[tid], 否则取 insts[tid]
 上限: min(来源队列大小, dispatchWidth)

对每条指令 (最多上限条):

```

├─ 若 isSquashed():
    |   └─ 丢弃该指令, 更新分发计数器, 继续

```

```

├─ 若 instQueue.isFull(inst) :
│   └─ block(tid), 跳出循环
├─ 若 LSQ 已满 (加载检查 LQ, 存储/原子检查 SQ) :
│   └─ block(tid), 跳出循环
├─ 按指令类型路由 :
│   └─ 原子操作 (Atomic) :
│       ├── ldstQueue.insertStore(inst) ← 使用 SQ 条目
│       ├── inst->setCanCommit()
│       ├── instQueue.insertNonSpec(inst)
│       └─ add_to_iq = false
│   └─ 加载 (Load) :
│       ├── ldstQueue.insertLoad(inst)
│       └─ add_to_iq = true ← 还需要插入 IQ 跟踪依赖
│   └─ 存储 (Store) :
│       ├── ldstQueue.insertStore(inst)
│       ├── 若是 StoreConditional :
│       │   ├── inst->setCanCommit()
│       │   ├── instQueue.insertNonSpec(inst)
│       │   └─ add_to_iq = false
│       └─ 否则 : add_to_iq = true
│   └─ 读屏障 / 写屏障 (ReadBarrier / WriteBarrier) :
│       ├── inst->setCanCommit()
│       ├── instQueue.insertBarrier(inst)
│       └─ add_to_iq = false
│   └─ 空操作 (Nop) :
│       ├── inst->setIssued(), setExecuted(), setCanCommit()
│       ├── instQueue.recordProducer(inst) ← 注册为生产者但不入队
│       └─ add_to_iq = false
│   └─ 其他 (非内存、非屏障、非 Nop) :
│       └─ add_to_iq = true
│           若是非推测指令 (isNonSpeculative) :
│               ├── inst->setCanCommit()
│               ├── instQueue.insertNonSpec(inst)
│               └─ add_to_iq = false
├─ 若 add_to_iq 为真 :
│   └─ instQueue.insert(inst)
├─ 从来源队列弹出
├─ 递增分发计数器
└─ 记录 dispatchTick 延迟统计

```

循环结束后 :

```

├─ 若来源队列非空 (带宽用尽) : block(tid)
└─ 若原状态为 Idle 且分发了指令 : status = Running

```

5.3 指令路由汇总

指令类型	LSQ 操作	IQ 操作	设置 canCommit?
加载 (Load)	<code>insertLoad()</code>	<code>insert()</code>	否
存储 (普通 Store)	<code>insertStore()</code>	<code>insert()</code>	否
条件存储 (StoreConditional)	<code>insertStore()</code>	<code>insertNonSpec()</code>	是
原子操作 (Atomic)	<code>insertStore()</code>	<code>insertNonSpec()</code>	是
读/写屏障	—	<code>insertBarrier()</code>	是
非推测指令	—	<code>insertNonSpec()</code>	是
空操作 (Nop)	—	<code>recordProducer()</code>	是
其他普通指令	—	<code>insert()</code>	否

5.4 阻塞条件

满足以下任一条件时，Dispatch 进入阻塞状态：

- 1. **IQ 已满**： `instQueue.isFull(inst)` — 指令的 OpClass 没有空闲 IQ 条目
- 2. **LQ 已满**： `ldstQueue.lqFull(tid)` — 该线程没有空闲加载条目
- 3. **SQ 已满**： `ldstQueue.sqFull(tid)` — 该线程没有空闲存储条目
- 4. **ROB 正在 Squash**： `fromCommit->commitInfo[tid].robSquashing` — Commit 阶段仍在清除中
- 5. **带宽用尽**：已达到 `dispatchWidth` 上限但还有剩余指令

阻塞时执行的操作：

- `dispatchStatus[tid]` 设为 `Blocked`
- `toRename->iewBlock[tid] = true` (通知 Rename 阶段停止发送)
- 当前 `insts[tid]` 中的指令全部移入 `skidBuffer[tid]` (通过 `skidInsert()`)

5.5 Skid Buffer (滑动缓冲区)

- **容量**： `(renameToIEWDelay + 1) * renameWidth`
- **作用**：当 Dispatch 阻塞时暂存来自 Rename 的流水线中指令，防止数据丢失
- **行为**：解阻塞时优先从 skid buffer 分发指令；在 Unblocking 转换期间，Rename 传来的新指令也会被追加到 skid buffer

5.6 HTM (硬件事务内存) 跟踪

在分发过程中，如果 LSQ 存在活跃的事务 (`numHtmStarts - numHtmStops > 0`)，则为指令标记当前 HTM UID 和事务深度。这确保 Commit 阶段能按程序顺序处理事务状态。

6. Execute (执行) 子阶段

6.1 executeInsts() 算法

重置 : `wbNumInst = 0, wbCycle = 0, fetchRedirect[tid] = false` (所有线程)

遍历每条待执行指令 (数量 = `fromIssue->size`) :

└─ `inst = instQueue.getInstToExecute()` // 从 `instsToExecute` 队列取出

└─ 触发 `ppExecute` 探测点

└─ 若 `isSquashed()` :

└─ `inst->setExecuted(), setCanCommit()`, 跳过该指令

└─ 若是内存引用指令 (`isMemRef`) :

└─ 原子操作 :

└─ `fault = ldstQueue.executeStore(inst)`

若翻译延迟且无 `fault` :

`instQueue.deferMemInst(inst)`, 跳过

└─ 加载 :

└─ `fault = ldstQueue.executeLoad(inst)`

若翻译延迟且无 `fault` :

`instQueue.deferMemInst(inst)`, 跳过

若为预取指令 : `inst->fault = NoFault`

└─ 存储 :

└─ `fault = ldstQueue.executeStore(inst)`

若翻译延迟且无 `fault` :

`instQueue.deferMemInst(inst)`, 跳过

若有 `fault` 或谓词为假或非条件存储 :

`inst->setExecuted(), instToCommit(inst)`,

`activityThisCycle()`

└─ 否则 (非内存指令) :

└─ 若 `inst->getFault() == NoFault` :

`inst->execute()` // 调用 ISA 层面的 `execute` 方法

若谓词为假 : `inst->forwardOldRegs()` // 转发旧寄存器值

`inst->setExecuted()`

`instToCommit(inst)` // 发送到 `Commit`

└─ 更新执行统计信息

└─ 检查是否需要重定向 (按线程) :

条件 : 未发生重定向 或 无 `pending squash` 或 `squash` 序号 > 当前指令序号

└─ 若 `inst->mispredicted()` 且不是 (未执行的加载) :

└─ `squashDueToBranch(inst, tid)`

触发 `ppMispredict` 探测点

更新 `predictedTakenIncorrect` / `predictedNotTakenIncorrect`

└─ 否则若 `ldstQueue.violation(tid)` :

```
└─ squashDueToMemOrder(violator, tid)
    instQueue.violation(inst, violator)
```

6.2 内存指令与非内存指令执行对比

对比项	非内存指令	内存指令 (Load/Store/Atomic)
执行方式	IEW 直接调用 <code>inst->execute()</code>	委托给 <code>ldstQueue.executeLoad/Store()</code>
结果处理	立即标记 <code>executed</code> ，发送到 <code>Commit</code>	LSQ 处理 TLB 翻译、缓存访问、响应接收
完成时机	同周期完成	异步完成，通过 LSQ 加载/写回事件
Fault 处理	执行前检查	由 LSQ 返回，在 <code>Commit</code> 阶段处理

6.3 分支预测错误检测

执行控制流指令后：

```
若 inst->mispredicted() :
    └─ 比较预测目标 (inst->readPredTarg()) 与实际目标 (inst->pcState())
        若不匹配：
            squashDueToBranch(inst, tid) :
                toCommit->squash[tid] = true
                toCommit->squashedSeqNum[tid] = inst->seqNum
                toCommit->pc[tid] = inst->pcState()
                inst->staticInst->advancePC(*toCommit->pc[tid]) // 计算正确
PC
                toCommit->mispredictInst[tid] = inst
                toCommit->includeSquashInst[tid] = false // 不包含该指令本身
```

优先级规则：如果该线程已有 `fetchRedirect`，且存在活跃的 `squash` 且其 `squashedSeqNum <= 当前指令序号`，则新的预测错误被抑制。

6.4 内存序违规检测

每条指令执行后检查：

```
若 ldstQueue.violation(tid) :
    └─ violator = ldstQueue.getMemDepViolator(tid)
        squashDueToMemOrder(violator, tid) :
            toCommit->squash[tid] = true
            toCommit->squashedSeqNum[tid] = violator->seqNum
            toCommit->pc[tid] = violator->pcState()
            toCommit->includeSquashInst[tid] = true // 将违规指令本身包含在
squash 范围内
```

与分支预测错误的区别：`includeSquashInst = true` 表示违规指令本身也需要被 squash 并重新执行。

6.5 重定向优先级

同一周期内每个线程只处理一次重定向：

- 1. **先发生者优先：**一旦 `fetchRedirect[tid]` 被设置，后续重定向被抑制（除非已有 squash 的 `squashedSeqNum` 更年轻）
- 2. **内存序违规 vs 分支预测错误：**当同一指令同时触发两者时，内存序违规优先，因为它需要将违规指令包含在 squash 范围内

7. 指令队列（Instruction Queue, IQ）

7.1 IQ 架构

```
InstructionQueue
├─ iqs: vector<IQUnit*>           // 多个 IQ 单元（每个关联独立的 FU 池）
├─ instList[MaxThreads]: list     // 所有已分发的指令
├─ instsToExecute: list           // 已调度待执行的指令
├─ readyInsts[Num_OpClasses]: priority_queue // 按操作类型分类的就绪队列
├─ listOrder: list<ListOrderEntry> // 按年龄排序的操作类型队列列表
├─ nonSpecInsts: map<SeqNum, DynInstPtr> // 非推测指令
├─ deferredMemInsts: list         // 等待 DTB 翻译的内存指令
├─ blockedMemInsts: list         // 缓存阻塞的内存指令
├─ retryMemInsts: list           // 可重试的内存指令
├─ dependGraph: DependencyGraph  // 生产者-消费者依赖图
├─ regScoreboard: vector<bool>   // IQ 本地寄存器就绪状态缓存
└─ memDepUnit[MaxThreads]: MemDepUnit // 内存依赖预测单元
```

7.2 IQUnit（每池资源管理）

每个 IQUnit 管理指令队列的一个子集，配有独立的 FU 池：

字段	描述
<code>_numEntries</code>	该 IQUnit 的总条目数
<code>_freeEntries</code>	当前可用条目数
<code>maxEntries[MaxThreads]</code>	每线程最大配额（执行 SMT 共享策略）
<code>count[MaxThreads]</code>	每线程当前使用量
<code>_fuPool</code>	关联的功能单元池

SMT 共享策略：

- **Dynamic（动态）：**任何线程可使用全部条目
- **Partitioned（分区）：**条目平均分配给各线程

- **Threshold (阈值)**：每线程上限为总条目的指定百分比

7.3 指令插入流程

```
insert(new_inst):
|
|— instList[tid].push_back(new_inst)
|— IQUnit->insert(new_inst) // 减少空闲条目
|— addToDependents(new_inst) // 注册为消费者（建立源寄存器依赖）
|— addToProducers(new_inst) // 注册为生产者（设置目的寄存器生产者）
|— 若是内存指令：
|   |— memDepUnit[tid].insert(new_inst)
|— addIfReady(new_inst) // 若所有源操作数已就绪，移入就绪队列
```

7.4 依赖图 (DependencyGraph)

DependencyGraph 是一个链表数组，每个物理寄存器对应一条链表：

```
dependGraph[物理寄存器号]:
  头节点 → 生产者指令（写入该寄存器的指令）
  后续节点 → 消费者1 → 消费者2 → ... → 消费者N

操作：
  insert(reg_idx, inst)    — 将消费者添加到 reg_idx 链表头部
  setInst(reg_idx, inst)   — 设置 reg_idx 的生产者指令
  pop(reg_idx)             — 移除并返回最新的消费者
  remove(reg_idx, inst)    — 从链表中移除指定消费者
  clearInst(reg_idx)       — 清除 reg_idx 的生产者
```

addToDependents()：遍历新指令的每个源寄存器：

- 若 **regScoreboard[flatIndex]** 为 false：插入依赖图
- 若 **regScoreboard[flatIndex]** 为 true：标记该源寄存器已在指令上就绪

addToProducers()：遍历新指令的每个目的寄存器：

- 断言该寄存器的依赖图为空（不存在未完成的生产者）
- 设置指令为该寄存器的生产者（头节点）
- 标记 **regScoreboard[flatIndex] = false**

7.5 就绪指令调度

7.5.1 年龄序列表 (listOrder)

listOrder 维护就绪指令队列之间的相对年龄排序：

```
listOrder: [OpClass=IntAdd, 最老序号=42] → [OpClass=IntMul, 最老序号=45] →
[OpClass=FloatAdd, 最老序号=50]
```

每个条目记录操作类型和该类型就绪队列中最老指令的序列号。这确保了跨所有操作类型的"最老优先"调度，防止指令饥饿。

7.5.2 scheduleReadyInsts() 算法

首先调度延迟/阻塞的内存指令：

```
|
|— getDeferredMemInstToExecute() → addReadyMemInst()
|— getBlockedMemInstToExecute() → addReadyMemInst()
|
```

遍历 listOrder 中每个条目（从最老开始），最多 totalWidth 条：

```
|
|— op_class = entry.queueType
|— inst = readyInsts[op_class].top() // 该类型中最老的指令
|
|— 若指令已被 squash：
|   |— 从就绪队列弹出，从 listOrder 擦除，继续
|
|— 从 IQUnit 的 fuPool 获取功能单元：
|   idx = fu_pool->getUnit(op_class)
|   op_latency = fu_pool->getOpLatency(op_class)
|
|— 若 FU 可用 (idx > NoFreeFU) 或不需要 FU (NoNeedFU/NoCapableFU)：
|   |
|   |— 若 op_latency == 1 (单周期)：
|   |   |— issueToExecuteQueue->access(-1)->size++
|   |   |— instsToExecute.push_back(inst)
|   |   |— fu_pool->freeUnitNextCycle(idx) 若 idx >= 0
|   |
|   |— 若 op_latency > 1 (多周期)：
|   |   |— ++wbOutstanding
|   |   |— event = new FUCompletion(inst, fu_pool, idx, this)
|   |   |— schedule(event, clockEdge(op_latency - 1))
|   |   |— 若非流水线：event->setFreeFU() // 完成时释放 FU
|   |       否则：fu_pool->freeUnitNextCycle(idx) // 下周期即可释放
|
|— inst->setIssued()
|— 若非内存指令：inst->clearInIQ() // 从 IQ 移除
|— 否则：memDepUnit[tid].issue(inst)
|— 从就绪队列弹出，从 listOrder 擦除
|
|— 若 FU 忙 (idx == NoFreeFU)：
|   |— ++fuBusy 统计，尝试下一个操作类型
```

7.5.3 FU 池分配

```

FUPool::getUnit(capability):
|
|— 若 capability 不在 capabilityList 中: 返回 NoCapableFU(-2)
|
|— idx = fuPerCapList[capability].getFU()
|   |— 从环形队列中获取可用 FU 索引
|
|— 若无空闲 FU: 返回 NoFreeFU(-1)
|
|— 标记 unitBusy[idx] = true
|
|— 返回 idx

```

FU 释放时机:

- 单周期操作: 通过 `freeUnitNextCycle()` 标记下周期释放
- 多周期流水线操作: 下周期释放 (可接受新发射)
- 多周期非流水线操作: 仅在 `FUCompletion` 事件触发后释放

7.5.4 FUCompletion 事件

```

FUCompletion::process():
|
|— processFUCompletion(inst, fu_pool, fu_idx):
|   |— --wbOutstanding
|   |— iewStage->wakeCPU() // 唤醒休眠的 CPU
|   |— fu_pool->freeUnitNextCycle(fu_idx) 若 fu_pool != nullptr
|   |— issueToExecuteQueue->access(-1)->size++
|   |— instsToExecute.push_back(inst)
|

```

`access(-1)` 写入的是下个周期将被读取的时间槽 (当前周期的未来槽位)。

7.6 唤醒依赖指令

```

wakeDependents(completed_inst):
|
|— 若是内存指令: memDepUnit[tid].completeInst(completed_inst)
|
|— 遍历每个目的寄存器:
|   |— 若是 isAlwaysReady (固定映射寄存器): 跳过
|   |— 递减 numPinnedWritesToComplete; 若仍 > 0: 跳过
|   |
|   |— while (dep = dependGraph.pop(dest_reg->flatIndex())):
|       |— dep->markSrcRegReady() // 标记该源寄存器就绪
|       |— addIfReady(dep)
|           |— 若 readyToIssue(): 推入就绪队列 readyInsts[op_class]
|
|— dependGraph.clearInst(dest_reg->flatIndex())

```

```
└─ regScoreboard[dest_reg->flatIndex()] = true
└─ 返回被唤醒的依赖指令数量
```

7.7 非推测指令

非推测指令（屏障、条件存储、原子操作）保存在 `nonSpecInsts` 映射中，不主动调度，直到 Commit 发送 `nonSpecSeqNum` 信号：

```
scheduleNonSpec(seqNum) :
|
└─ inst = nonSpecInsts[seqNum]
└─ inst->setAtCommit()
└─ inst->setCanIssue()
└─ 若非内存指令：addIfReady(inst)
   否则：memDepUnit[tid].nonSpecInstReady(inst)
└─ 从 nonSpecInsts 中擦除
```

7.8 内存指令状态管理

状态链表	触发条件	解决方式
<code>deferredMemInsts</code>	DTB 翻译延迟（硬件页表遍历）	翻译完成 → <code>getDeferredMemInstToExecute()</code>
<code>blockedMemInsts</code>	缓存阻塞（无响应）	缓存重试/解阻塞 → 移入 <code>retryMemInsts</code>
<code>retryMemInsts</code>	缓存可用，可重试	<code>getBlockedMemInstToExecute()</code> → <code>addReadyMemInst()</code>

7.9 Squash 处理

```
squash(tid) :
|
└─ squashedSeqNum[tid] = fromCommit->commitInfo[tid].doneSeqNum
└─ doSquash(tid)
└─ memDepUnit[tid].squash(squashedSeqNum[tid], tid)

doSquash(tid) :
|
└─ 从 instList[tid] 尾部（最年轻）向头部遍历
   对每条 seqNum > squashedSeqNum[tid] 的指令：
   |
   └─ 若已标记 squashedInIQ：跳过
   |
   └─ 若未发射 或（内存指令且未完成）：
       └─ 从依赖图中移除（消费者条目）
       └─ 若是非推测指令：从 nonSpecInsts 移除
       └─ 标记 setSquashedInIQ(), setIssued(), setCanCommit(),
```

```
clearInIQ()  
├── ┌── 清除依赖图中的生产者条目  
│   └──  
└── ┌── 从 instList[tid] 中擦除  
    └──
```

7.10 Commit 处理

```
commit(seqNum, tid):  
└── 从 instList[tid] 中移除所有 seqNum <= seqNum 的指令  
    (这些指令已被 Commit 阶段退休, 不再需要跟踪)
```

8. 加载/存储队列 (LSQ)

8.1 LSQ 架构

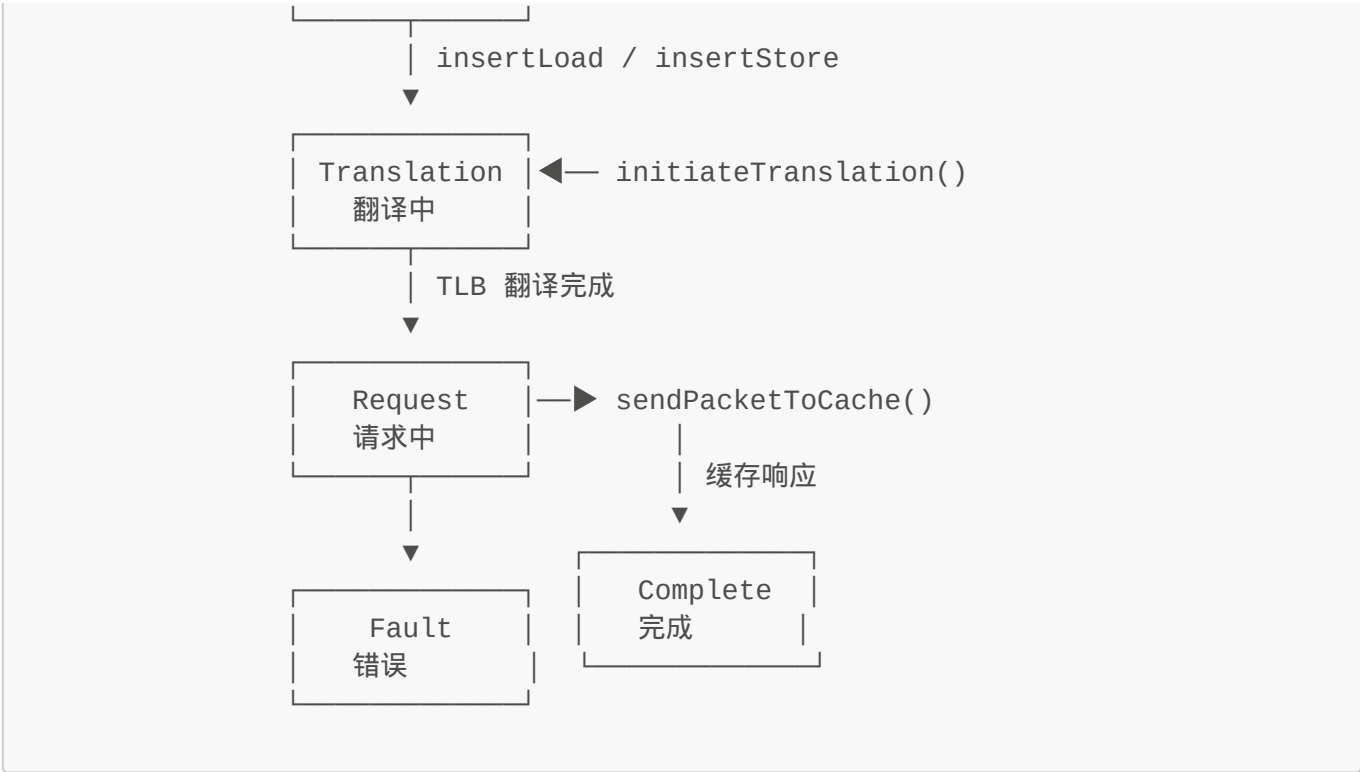
```
LSQ  
├── thread: vector<LSQUnit>           // 每线程 LSQ 单元  
├── dcachePort: DcachePort           // 数据缓存请求/响应端口  
├── LQEntries: unsigned               // 加载队列总条目数  
├── SQEntries: unsigned               // 存储队列总条目数  
├── _cacheBlocked: bool               // D-cache 阻塞标志  
├── cacheLoadPorts: int               // 每周期可用加载端口数  
├── cacheStorePorts: int              // 每周期可用存储端口数  
├── usedLoadPorts: int                // 本周期已用加载端口数  
├── usedStorePorts: int               // 本周期已用存储端口数  
└── recvRespThrottling: bool         // 加载响应流量限制
```

8.2 LSQUnit (每线程单元)

```
LSQUnit  
├── loadQueue: deque<LQEntry>         // 加载条目队列  
├── storeQueue: deque<SQEntry>        // 存储条目队列  
├── memDepViolator: DynInstPtr        // 内存序违规者指令  
├── cacheBlocked: bool                // 本地缓存阻塞标志  
├── storeWaitingOnWB: bool             // 存储等待写回  
├── loadWaitingOnWB: bool              // 加载等待写回  
└── loadBlockedOnStore: bool           // 加载被更早存储阻塞
```

8.3 LSQRequest 状态机





LSQRequest 标志位:

标志	含义
IsLoad	请求为加载操作
WriteBackToRegister	需要写回寄存器（加载或条件存储）
Delayed	翻译/访问延迟
IsSplit	跨缓存行分割
TranslationStarted	TLB 翻译已发起
TranslationFinished	TLB 翻译已完成
Sent	数据包已发送到缓存
Retry	发送失败，需重试
Complete	操作完成
TranslationSquashed	翻译已被 squash
Discarded	请求已丢弃
LSQEntryFreed	LSQ 资源已释放
WritebackScheduled	存储写回已调度
WritebackDone	存储写回已完成
IsAtomic	原子操作

8.4 内存操作流程

8.4.1 加载指令



8.4.2 存储指令



- |— TLB 地址翻译
- |— 发送 D-cache 原子操作请求
- |— 响应：将结果写回寄存器，完成操作

8.5 内存序违规检测

LSQ 检测加载指令在更早的存储指令之前执行并获取了错误数据的违规范例：

违规检测：

- |— 检查是否有已执行的加载从更年轻的存储获取了数据
（加载地址匹配存储地址，存储更年轻）
- |— 若检测到违规：
 - |— memDepViolator = 本应在加载之前完成的存储指令
 - |— 返回 true

squashDueToMemOrder：

- |— 从违规指令的 seqNum 开始 squash（包含违规指令本身）
- |— 重新执行：加载现在将看到存储的正确数据

8.6 存储写回

writebackStores()：

- |— 遍历每个有已提交待写回存储的线程：
 - |— 若有缓存端口可用：
 - |— 发送存储数据到 D-cache
 - |— 响应后：释放 SQ 条目
 - |— 若缓存阻塞：停止

8.7 LSQ 提交处理

commitStores(seqNum, tid)：

- |— 标记所有 seqNum <= seqNum 的存储条目为已提交
（可向内存写回）

commitLoads(seqNum, tid)：

- |— 标记所有 seqNum <= seqNum 的加载条目为已提交
（可释放资源）

9. Writeback（写回）子阶段

9.1 writebackInsts() 算法

```

遍历 inst_num = 0 到 wbWidth - 1:
|
|— inst = toCommit->insts[inst_num]  // 从 iewQueue 中获取
|   └─ 若为空：跳出循环
|
|— 触发 ppToCommit 探测点
|
|— 若指令未被 squash 且已执行且无 fault:
|   |
|   |— dependents = instQueue.wakeDependents(inst)
|   |
|   |— 遍历每个目的寄存器:
|       └─ 若 numPinnedWritesToComplete == 0:
|           scoreboard->setReg(dest_reg)  // 标记物理寄存器就绪
|
|   └─ iewStats.writebackCount[tid]++

```

9.2 关键特性

- 写回处理来自 `iewQueue` 时间缓冲的指令，这些指令在执行阶段通过 `instToCommit()` 发送
- `wbWidth` 参数限制每周期可写回的指令数量
- 写回是**记分板更新**和**依赖唤醒**的关键点——这是寄存器就绪广播的核心环节
- Pinned 寄存器（如某些 CSR 写入）在所有待完成写入完成之前跳过记分板更新

9.3 instToCommit() — 发送到 Commit

```

instToCommit(inst):
|
|— 在 iewQueue 中查找空闲槽位:
|   └─ while iewQueue[wbCycle].insts[wbNumInst] 已被占用:
|       ++wbNumInst
|       若 wbNumInst == wbWidth: ++wbCycle, wbNumInst = 0
|
|— iewQueue[wbCycle].insts[wbNumInst] = inst
|— iewQueue[wbCycle].size++

```

若单周期内完成的指令超过 `wbWidth` 条，溢出部分会写入 `iewQueue` 的未来时间槽位，实质上推迟到后续周期写回。

10. 记分板（Scoreboard）

10.1 作用

记分板跟踪哪些物理寄存器的值已就绪（已写回）。IQ 在 `addToDependents()` 中查询记分板，判断新指令的源寄存器是否可用。

10.2 接口

```
class Scoreboard {
    bool getReg(PhysRegIdPtr);    // 寄存器是否就绪？
    void setReg(PhysRegIdPtr);    // 标记寄存器就绪（写回时调用）
    void unsetReg(PhysRegIdPtr);  // 标记寄存器忙（重命名时调用）
};
```

10.3 与 IQ 本地记分板的交互

IQ 维护自己的 `regScoreboard` 向量，与全局记分板保持镜像关系：

- **全局记分板**（`scoreboard`）：IEW 写回阶段更新
- **IQ 本地记分板**（`regScoreboard`）：IQ 的 `wakeDependents()` 中更新

IQ 本地副本避免了指令插入时访问全局记分板的开销，但两者必须保持一致。

10.4 始终就绪寄存器

满足 `isAlwaysReady() == true` 的寄存器（固定映射寄存器，如零寄存器、某些 CSR）绕过记分板跟踪：

- 从不插入依赖图
- 从不标记为忙/未就绪
- 作为源操作数时始终视为可用

11. Squash（清除）处理

11.1 Squash 来源

来源	触发条件	处理函数
分支预测错误	Execute 检测到预测结果与实际不符	<code>squashDueToBranch()</code>
内存序违规	LSQ 检测到加载-存储顺序违规	<code>squashDueToMemOrder()</code>
Commit 发起的 Squash	Trap、异常、外部 squash	<code>checkSignalsAndUpdate()</code>
ROB 正在 Squash	Commit 仍在处理之前的 squash	<code>checkSignalsAndUpdate()</code>

11.2 IEW Squash 响应流程

```
checkSignalsAndUpdate(tid) :
|
|— 若 fromCommit->squash 为真：
|   |— squash(tid)
|       |— instQueue.squash(tid)
```

```

└─ ldstQueue.squash(doneSeqNum, tid)
└─ 清空 skidBuffer[tid]
└─ emptyRenameInsts(tid)
└─ dispatchStatus[tid] = Squashing

└─ 若 fromCommit->robSquashing 为真：
    └─ dispatchStatus[tid] = Squashing
        emptyRenameInsts(tid)

└─ 若 checkStall(tid) 为真 (IQ 满 / LSQ 满 / ROB 正在 squash)：
    └─ block(tid), dispatchStatus[tid] = Blocked

└─ 若原状态为 Blocked 且不再阻塞：
    └─ dispatchStatus[tid] = Unblocking, unblock(tid)

└─ 若原状态为 Squashing 且 squash 已清除：
    └─ dispatchStatus[tid] = Running

```

11.3 Squash 级联传播

IEW 中发起 squash 时的级联操作：

```

IEW::squash(tid)
└─ instQueue.squash(tid)
    └─ 清除所有比 squashedSeqNum 更年轻的指令

└─ ldstQueue.squash(doneSeqNum, tid)
    └─ 清除所有比 doneSeqNum 更年轻的加载/存储条目

└─ 清空 skidBuffer[tid] (更新分发计数器)

└─ emptyRenameInsts(tid) (排空来自 Rename 的待处理指令)

└─ 通过时间缓冲发送反向信号：
    └─ toCommit->squash[tid] = true (用于分支/内存违规)
    └─ toRename->iiewUnblock[tid] = true (若之前被阻塞)

```

12. 状态更新与功耗管理

12.1 updateStatus()

```

updateStatus():
└─ any_unblocking = 是否存在 dispatchStatus == Unblocking 的线程

└─ 若 Active 且 !hasReadyInsts 且 !ldstQueue.willWB() 且 !any_unblocking:
    └─ → 切换为 Inactive (停用阶段, CPU 可进入非调度低功耗状态)

```

|

└ 若 Inactive 且 (hasReadyInsts 或 ldstQueue.willWB() 或 any_unblocking) :
└ ─→ 切换为 Active (激活阶段)

12.2 活跃度信号

以下情况 IEW 阶段向 CPU 发送活跃度信号：

- 从 Rename 分发指令
- 执行指令
- 写回发生
- IQ 调度就绪指令
- 缓存解阻塞
- 发起 Squash

发送活跃度信号时调用 `cpu->activityThisCycle()`，防止 CPU 进入低功耗非调度状态。

13. 配置参数

13.1 IEW 阶段参数

参数	类型	默认值	描述
<code>dispatchWidth</code>	Unsigned	4	每周期最大分发到 IQ/LSQ 的指令数
<code>issueWidth</code>	Unsigned	8	每周期从 IQ 发射的最大指令数
<code>wbWidth</code>	Unsigned	3	每周期最大写回指令数
<code>renameToIEWDelay</code>	Cycles	2	Rename 到 IEW 的流水线延迟
<code>commitToIEWDelay</code>	Cycles	1	Commit → IEW 反向信号延迟
<code>issueToExecuteDelay</code>	Cycles	1	IQ 调度到执行的内部延迟
<code>iewToFetchDelay</code>	Cycles	1	IEW → Fetch 反向信号延迟
<code>iewToDecodeDelay</code>	Cycles	1	IEW → Decode 反向信号延迟
<code>iewToRenameDelay</code>	Cycles	1	IEW → Rename 反向信号延迟
<code>iewToCommitDelay</code>	Cycles	1	IEW → Commit 正向信号延迟

13.2 IQ 参数（通过 `instQueues`）

参数	描述
<code>numEntries</code>	每个 IQUnit 的总条目数
<code>smtIQPolicy</code>	SMT 共享策略：Dynamic / Partitioned / Threshold
<code>smtIQThreshold</code>	Threshold 模式下每线程最大利用百分比

参数	描述
fuPool	关联的 FUPool

13.3 LSQ 参数

参数	描述
LQEntries	加载队列总条目数
SQEntries	存储队列总条目数
lsqPolicy	SMT 共享策略：Dynamic / Partitioned / Threshold
SMTThreshold	Threshold 模式下每线程最大配额

13.4 FUPool 参数

参数	描述
opClasses	定义 FU 能力的 OpClass 条目列表
opLatencies	每个 OpClass 的操作延迟（周期数）
pipelined	每个 OpClass 是否支持流水线执行

14. 统计信息

14.1 IEW 统计

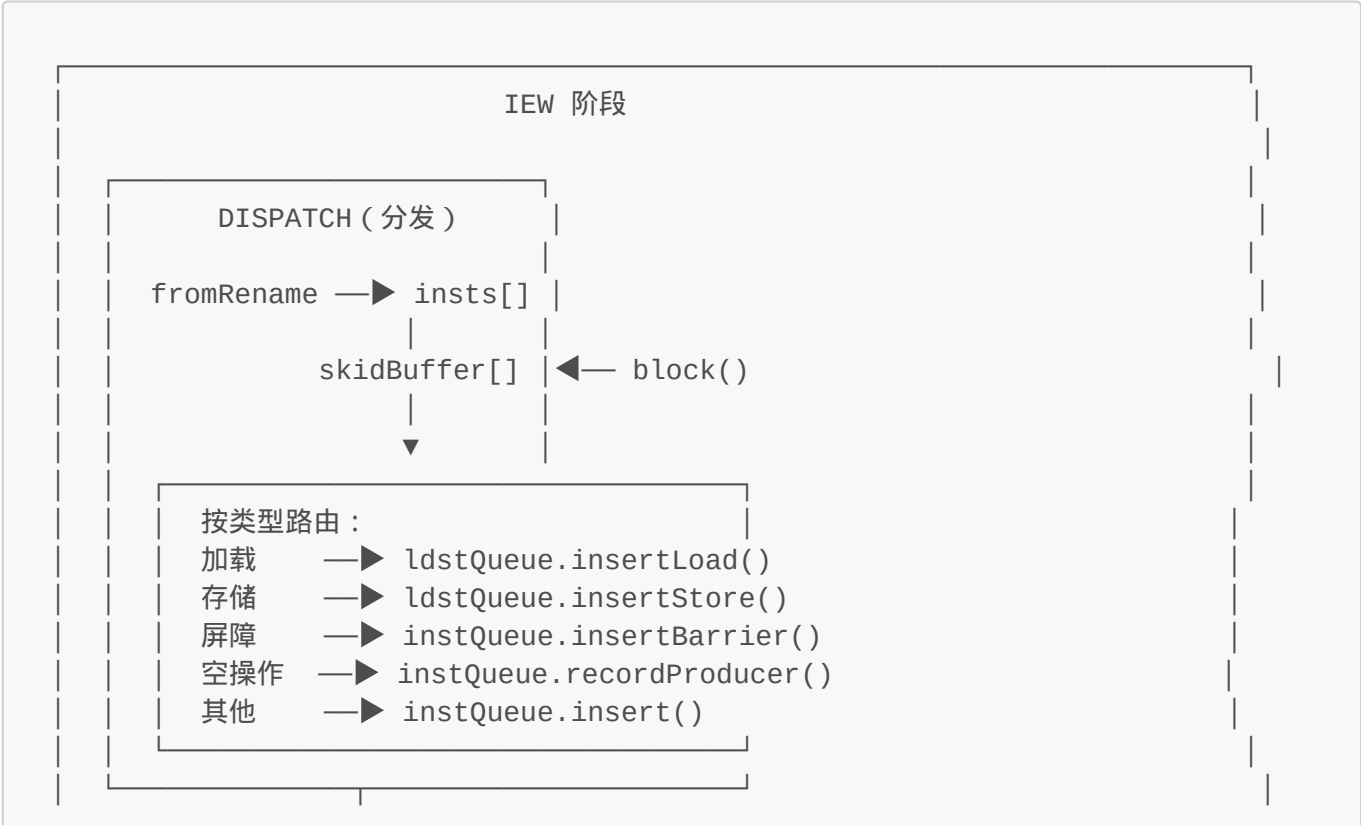
统计项	单位	描述
dispatchStatus	周期	在各分发状态下花费的周期数
execStatus	周期	在各执行状态下花费的周期数
dispatchedInsts	计数	分发到 IQ 的指令总数
dispSquashedInsts	计数	Dispatch 跳过的已 squash 指令数
dispLoadInsts	计数	分发的加载指令数
dispStoreInsts	计数	分发的存储指令数
dispNonSpecInsts	计数	分发的非推测指令数
iqFullEvents	计数	IQ 满导致的停顿事件数
lsqFullEvents	计数	LSQ 满导致的停顿事件数
memOrderViolationEvents	计数	内存序违规事件数
branchMispredicts	公式	分支预测错误总数
predictedTakenIncorrect	计数	预测为执行但实际未执行的分支数

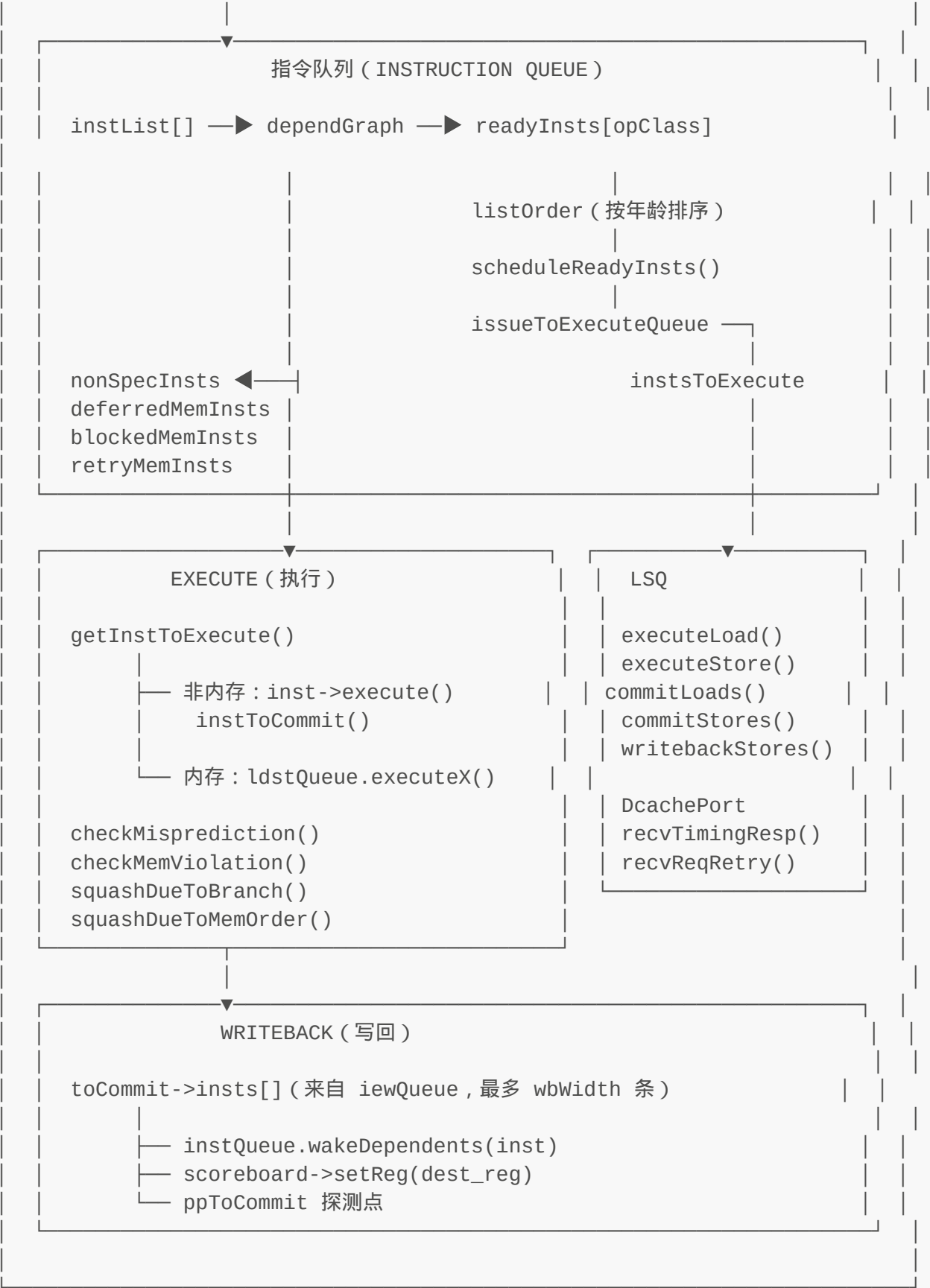
统计项	单位	描述
predictedNotTakenIncorrect	计数	预测为不执行但实际执行的分支数
instsToCommit	计数	发送到 Commit 的指令数（每线程）
writebackCount	计数	写回的指令数（每线程）
producerInst	计数	唤醒了依赖的指令数
consumerInst	计数	被唤醒的依赖指令总数
wbRate	公式	每周期写回指令数
wbFanout	公式	每次写回平均唤醒的依赖数

14.2 IQ 统计

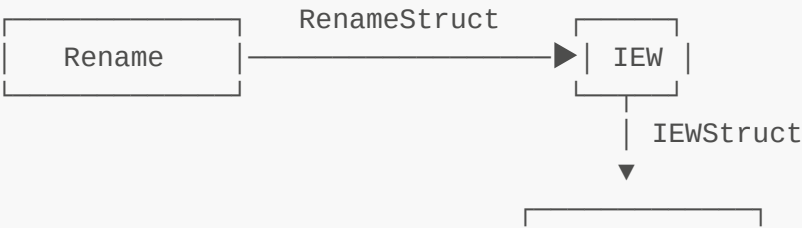
统计项	描述
instsAdded	添加到 IQ 的指令数（不含非推测指令）
nonSpecInstsAdded	添加到 IQ 的非推测指令数
instsIssued	发射的指令总数
numIssuedDist	每周期发射指令数的分布
fuBusy	每线程的 FU 忙事件数
fuBusyRate	FU 忙比率（忙事件数 / 执行指令数）

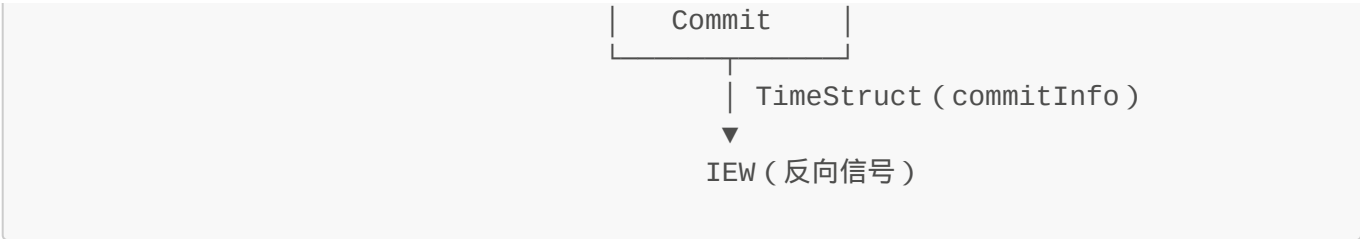
15. 数据流图





外部连接：





16. 附录

A. OpClass 系统

指令按 **OpClass** 类型分类，用于 IQ 调度和 FU 分配。每个 OpClass 映射到特定的 FU 能力。IQ 为每个 OpClass 维护独立的 **readyInsts** 优先级队列，实现类型感知的调度。

B. TimeBuffer 时间缓冲机制

gem5 使用 **TimeBuffer<T>** 进行阶段间通信。关键特性：

- **访问过去** (**access(-N)**)：读取 N 个周期前的数据
- **访问未来** (**access(+N)**)：写入 N 个周期后的数据
- **advance()**：将所有时间槽向前推进一个周期
- **Wire 连线**：封装固定偏移延迟的访问器

C. SMT 线程调度

所有 IEW 操作遍历 **activeThreads** 链表而非固定线程索引。这确保只有活跃（非休眠）线程消耗带宽和资源。

D. 适用版本

本规范基于 gem5 v25.1.0.0，源代码路径为 **src/cpu/o3/**。